

Chapter 11

Online learning

In this chapter, we switch gears and talk about *online learning* and *online convex optimization*. The main idea driving online learning is that we move away from the assumption that the training and test data are both drawn i.i.d from some fixed distribution. In the online setting, training data and test data come to the user in an interwoven manner, and data can be generated *adversarially*. We will describe how online learning can be reduced to online convex optimization, some important algorithms, as well as applications of these algorithms to some illustrative examples.

11.1 Online learning setup

In classical supervised learning, we train the model with the assumption that $(x^{(i)}, y^{(i)}) \stackrel{i.i.d.}{\sim} P_{\text{train}}$, where P_{train} is the underlying distribution of the training data. In most cases, we assume the test data, i.e., the data we want our model to predict well, comes from the same distribution (or at least one that is close to P_{train}). Reality is often more complicated: data could indeed be generated in sequence, or even in an adversarial manner, so it is often the case that P_{test} differs from P_{train} . The situation where P_{test} and P_{train} are different is known as *domain shift*. There are some theories that tackle the issue of domain shift and generalization properties of transfer learning. However, the field is still largely being developed. (See Ben-David et al. 2007, for example.)

Online learning is an attempt to deal with domain shift in a way that is agnostic to the relationship between the training and test data distributions (i.e. deal with “worst-case” domain shift). As an example, many recommendation systems today collect users’ historical trace of shopping behavior, which are not i.i.d. samples, and makes adaptive recommendations based on users’ changing shopping behavior. Hence, one can see that online learning attempts to adapt to the constantly evolving reality on time. Notice that unlike the “offline model” (i.e., classical supervised learning), online learning learns while testing, and hence there is no rigid division in time to differentiate training and testing phase.

Online learning has several distinctive features Liang, 2016:

1. The data may be *adversarial*. We cannot assume that sample is drawn independently from some distribution.
2. The data and predictions are *sequential*. At each step, the algorithm makes a prediction after given a single piece of data.
3. The feedback is *limited*. For example, in bandit problems, the algorithm only knows if its right or wrong, but no other feedback is given.

Online learning can be viewed as a game between two parties: (i) the learner/agent/algorithm/player, and (ii) the environment/nature. For simplicity, we will refer to the two parties as “learner” and “environment” in the remainder of this chapter.

The game takes place over T rounds or time steps. At each step $t = 1, \dots, T$, the learner receives an input $x_t \in \mathcal{X}$ from the environment and makes a prediction $\hat{y} \in \mathcal{Y}$ in response. The learner then receives the label y_t from the environment and suffers some loss. This procedure is outlined in Algorithm 1 and is illustrated in Figure 11.1

Algorithm 1: General online learning problem

```

1 for  $t = 1, \dots, T$  do
2   Learner receives  $x_t \in \mathcal{X}$  from environment, which may be chosen adversarially;
3   Learner predicts  $\hat{y} \in \mathcal{Y}$ ;
4   Learner receives the label  $y_t$ , from environment, which may be chosen adversarially; Learner
    suffers some loss  $\ell(y_t, \hat{y}_t)$ .

```

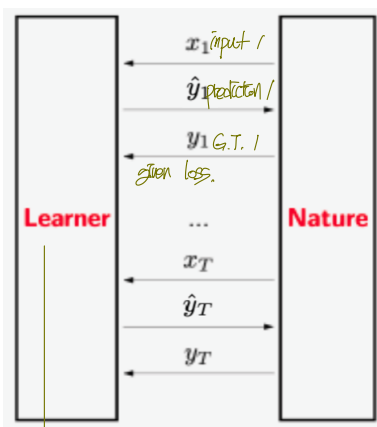


Figure 11.1: A representation of the online learning problem.

Later, we will see that the manner in which nature generates (x_t, y_t) leads to different types of online learning. In the most adversarial setting of online learning, it is possible that the “true label” y_t is not generated at the same time as x_t . The environment could generate the label y_t depending on the prediction $\hat{y}_t$ made by the learner. We can also see that Algorithm 1 is a very general framework as there are very few constraints on how x_t and y_t are generated.

11.1.1 Evaluation of the learner

Given this setup, a natural question to ask is how one can evaluate the performance of the learner. Intuitively, one could simply evaluate the learner’s performance by computing the loss between the predicted label and the “true” label sent by the environment $\ell(y_t, \hat{y}_t)$. For the entire sequence of tasks, one can then evaluate in terms of the cumulative loss:

$$\sum_{t=1}^T \ell(y_t, \hat{y}_t). \quad (11.1)$$

However, as the environment can be adversarial, the task itself might be inherently hard and even the best possible learner fails to achieve a small loss. Hence, instead of using the cumulative loss for a learner by itself, we compare its performance against a suitable baseline, the “best model in hindsight”. Assume that our learner comes from a set of hypotheses $\mathcal{H}$. Let us choose the hypothesis $h \in \mathcal{H}$ that minimizes the cumulative loss, i.e.

$$h^* = \operatorname{argmin}_{h \in \mathcal{H}} \sum_{t=1}^T \ell(y_t, h(x_t)). \quad (11.2)$$

optimal hypothesis. = best loss

best fixed model in hindsight

Regret function

Note here that in minimizing the **cumulative loss**, the learner gets to see all the data points (x_t, y_t) at once. The cumulative loss of h^* is the best we can ever hope to do, and so it would be better to compare the cumulative loss of the learner against it. (This approach is analogous to “excess risk”, which tells how far the current model is away from the best we could hope for.) This measurement is denoted as **regret**, and is formally defined as:

$$\text{Regret} \triangleq \left[\sum_{t=1}^T \ell(y_t, \hat{y}_t) \right] - \underbrace{\left[\min_{h \in \mathcal{H}} \sum_{t=1}^T \ell(y_t, h(x_t)) \right]}_{\text{best loss in hindsight}} \quad (11.3)$$

Using this definition, if the best model in hindsight performs well, then the learner has more responsibility to learn to predict well in order to match up the performance of the baseline.

11.1.2 The realizable case

In general, if the environment is too powerful, leading the learner to a large loss, it will also hinder the best model in hindsight from doing well. On the other hand, there are settings where some members of the hypothesis class can actually do well. Such settings/problems are usually referred to as **realizable**:

Definition 11.1 (Realizable problem). An online learning problem is **realizable** (for a family of predictors $\mathcal{H}$) if there exists $h \in \mathcal{H}$ such that for any T , $\sum_{t=1}^T \ell(y_t, h(x_t)) = 0$. “best loss $\rightarrow 0$ ”

Note that even though zero error is possible, this is still an interesting problem to consider because the x_t ’s are not i.i.d. as they are in classical supervised learning. Hence, standard statistical learning theory does not apply, and there is still research to be done here.

Example 11.2. Consider a classification problem on (x_t, y_t) , and for simplicity assume $y_t \in \{0, 1\}$. Suppose there exists $h^* \in \mathcal{H}$ such that we always have $y_t = \hat{y}_t^* = h^*(x_t)$. In this case, the problem is realizable.

In this case, the learner can adopt a **majority algorithm**. At each time, the learner maintains a set $V_t \subset \mathcal{H}$ so that $\sum_{t=1}^T \ell(y_t, h(x_t)) = 0$ for all $h \in V_t$, and $\hat{y}_t$ is simply the prediction made by the majority of $h \in V_t$. Based on the loss received, learners $h \in V_t$ that fail for time $t+1$ will be **eliminated** from future V_t ’s.

With this setup, we can see that for each wrong prediction made by the learner, at least half of the hypotheses $h \in V_t$ will be eliminated. Hence, $1 \leq |V_{t+1}| \leq |\mathcal{H}|2^{-M}$ where M is the number of mistakes made so far. Thus, one has $M \leq \log |\mathcal{H}|$ by taking log on both sides of inequalities and rearrange.

Now, if one puts ℓ as the **zero-one loss**, the regret for this example will be

$$\text{Regret} = \sum_{t=1}^T \ell(y_t, h(x_t)) = M, \leq \frac{\log |\mathcal{H}|}{\text{mistake bound}} \quad (11.4)$$

so in this example, one has $\text{regret} \leq \log |\mathcal{H}|$, which is a **non-trivial bound** when $\mathcal{H}$ is finite.

As one can see in the example, the realizable case usually indicates that the problem is not too far out of reach. Indeed, for finite hypothesis classes and linear models, the realizable case is considered to be straightforward to solve. This is perhaps why most of the past literature has focused on non-realizable cases. However, the realizable case is still an interesting problem and perhaps a very good starting point when the model class is beyond linear models and when the loss function is no longer convex, because the x_t ’s are not i.i.d. as they are in classical supervised learning. Hence, standard statistical learning theory does not apply, and there is still research to be done here.

In the rest of the chapter, we will only focus on the convex loss case, where we reduce online learning to online convex optimization.

11.2 Online (convex) optimization (OCO)

Online convex optimization (OCO) is a particularly useful tool to get results for online learning. Many online learning problems (and many other types of problems!) can be reduced to OCO problems, which allow them to be solved and analyzed algorithmically. Algorithm 2 describes the OCO problem, which is more general than the online learning problem. (Note: *Online optimization (OO)* refers to Algorithm 2 except that the f_t 's need not be convex. However, due to the difficulty in non-convex function optimization, most research has focused on OCO.)

Algorithm 2: Online (convex) optimization problem

- 1 for $t = 1, \dots, T$ do
- 2 The learner picks some action $w_t \in \Omega$ from the action space Ω ; : parameter
- 3 The environment picks a (convex) function $f_t : \Omega \rightarrow [0, 1]$;
- 4 The learner suffers the loss $f_t(w_t)$ and observes the entire loss function $f_t(\cdot)$. : $f_t(w_t)$ 만큼 loss $\rightarrow$ entire $f_t(\cdot)$ observes

Essentially the learner is trying to minimize the function f_t at each step. As with online learning, one evaluates the performance of learner in online optimization setting using the regret:

$$\text{Regret} = \sum_{t=1}^T f_t(w_t) - \underbrace{\min_{w \in \Omega} \sum_{t=1}^T f_t(w)}_{\text{best action in hindsight}} \quad (11.5)$$

: $f_t(w_t)$ 현재
best와 비교

At some level, OCO seems like an impossible task, since we are trying to minimize a function f_t that we only get to see after we have made our prediction! This is certainly the case for $t = 1$. However, as time goes on, we see more and more functions and, if future functions are somewhat related to past functions, we have more information to make better predictions. (And if the future functions are completely unrelated or contradictory to past functions, then the best action in hindsight would also be bad and therefore our algorithm does not have to do much.)

11.2.1 Settings and variants of OCO → f_t 선택의 문제

There are multiple settings of the OCO network, which can vary the power of the environment and observations.

- : i.i.d sampling
easier! ● Stochastic setting: $f_1, \dots, f_T$ are i.i.d samples from some distribution P . This corresponds to (x_t, y_t) being i.i.d. in online learning. Under this setting, the environment is not adversarial.
- : f_t 미리 알고 미리 대응
중립! ● Oblivious setting: $f_1, \dots, f_T$ are chosen arbitrarily but before the game starts. This corresponds to (x_t, y_t) being chosen before the game starts. In this setting, the environment can be adversarial but cannot be adaptive. The environment can choose these functions based on the learner's algorithm, but not the actual action if the learner's algorithm contains randomness. (This is the setting that we focus on in this course.)
- : 비~서로 알고 f_t 를 선택
hardest! ● Non-oblivious/adaptive setting: For all t , f_t can depend on the learner's actions $w_1, \dots, w_t$. Under this setting, the environment can be adversarial and adaptive. This is the most challenging setting because the environment is powerful enough to know not only the strategy of the learner, but also the exact choice the learner finally made. (Note however that If the learner is deterministic, the environment does not have more power here than in the oblivious setting. The oblivious adversary can simulate the game before the game starts, and chose the most adversarial input accordingly.)

11.3 Reducing online learning to online optimization

There is a natural way to reduce the online learning problem to online optimization, with respect to a specific type of model h_w parametrized by $w \in \Omega$. Recall that in online learning problem, the learner predicts y_t upon receiving x_t . If the learner possesses oracle to solve online optimization problem, the learner can consult the oracle to obtain w_t , the parameter of the model as in online optimization problem, and then predict $\hat{y}_t = h_{w_t}(x_t)$. *predict by solver.*

In the next two subsections, we give two examples of how an online learning problem can be reduced to an OCO problem.

11.3.1 Example: Online learning regression problem

Consider the regression model $h_w(x) = w^\top x$ parameterized by w in parameter space Ω with squared error loss ℓ . Here is the online learning formulation of the regression problem:

Algorithm 3: Online learning regression problem

- 1 **for** $t = 1, \dots, T$ **do**
 - 2 The learner receives $x_t \in \mathbb{R}^d$ from the environment; *input*
 - 3 The learner predicts $\hat{y}_t$;
 - 4 The environment selects y_t and sends it to the learner;
 - 5 The learner suffers loss $\ell(y_t, \hat{y}_t) = (y_t - \hat{y}_t)^2$.
-

This can be reduced to the OCO problem in the following way:

Algorithm 4: OCO formulation of regression problem

- 1 **for** $t = 1, \dots, T$ **do**
 - 2 The learner receives $x_t \in \mathbb{R}^d$ from the environment;
 - 3 The learner gives x_t to the OCO solver and obtains $w_t \in \mathbb{R}^d$; *(using oracle/solver)*
 - 4 The learner predicts $\hat{y}_t = h_{w_t}(x_t) = w_t^\top x_t$; *from OCO solver*
 - 5 The environment selects y_t and sends it to the learner;
 - 6 The learner suffers loss $(y_t - h_{w_t}(x_t))^2$;
 - 7 With (x_t, y_t) observed, the learner can reconstruct the loss function $f_t(w) = (y_t - h_w(x_t))^2$ and give it to the OCO solver. *★*
-

In this example, we have the following correspondence:

- f_t in online optimization $\leftrightarrow$ squared error loss functions for (x_t, y_t) .
- w_t in online optimization $\leftrightarrow$ parameters of the linear model h_{w_t} .

Since $h_w(\cdot)$ is linear, the corresponding squared error loss function f_t are convex, and so we have effectively reduced the online linear regression problem to an online convex optimization problem.

Notice that in the previous example, the loss function f_t actually depends on the label y_t , which demonstrates that the key challenge in online optimization is that the function f_t is unknown to the learner when the prediction $\hat{y}_t$ is made.

11.3.2 Example: The expert problem

Suppose we wish to predict tomorrow's weather and 10 different TV channels provide different forecasts. Which one should we follow? Formally, consider a finite hypothesis class $\mathcal{H}$, where each $h \in \mathcal{H}$ represents an expert, and we wish to choose a h_t wisely at each time step. For simplicity, we assume the prediction is

binary, i.e. $\hat{y} \in \{0, 1\}$, and suppose the loss function is 0-1 loss. (The problem can easily be generalized to more general predictions and losses.) The problem is outlined in Algorithm 5

Algorithm 5: The expert problem

- 1 **for** $t = 1, \dots, T$ **do**
 - 2 The learner obtains predictions from N experts;
 - 3 The learner chooses to follow prediction of one of the experts $i_t \in [N]$;
 - 4 The environment gives the learner the true value. The learner is thus able to learn the loss of each of the experts: $\ell_t \in \{0, 1\}^N$;
 - 5 The learner suffers the loss of the expert which was chosen: $\ell_t(i_t)$.
-

We want to design a method that chooses i_t for each step (line 3 in Algorithm 5) to minimize the regret:

$$\text{Regret} \triangleq \mathbb{E} \left[\sum_{t=1}^T \ell_t(i_t) - \underbrace{\min_{i \in [N]} \sum_{t=1}^T \ell_t(i)}_{\text{the best expert in hindsight}} \right], \quad (11.6)$$

where the expected value is over i_t , thus covering the case where the i_t 's could be random.

To make the expert problem amenable to reduction to OCO, we introduce idea of a continuous action space. Instead of choosing i_t from $\Omega = [N]$, the learner chooses a distribution p_t from the N -dimensional simplex $\Delta(N) = \{p \in \mathbb{R}^N : \|p\|_1 = 1, p \geq 0\}$. The learner then samples $i_t \sim p_t$. With this formulation, instead of selecting particular expert i_t to follow, the learner adjusts the belief p_t , and samples from the distribution to choose which expert to follow. Algorithm 6 outlines this procedure. Note that the loss is the expected loss $\mathbb{E}_{i \sim p_t}[\ell_t(i)]$ instead of the sampled $\ell_t(i_t)$. (확률 선택 문제)

Algorithm 6: The expert problem with continuous action

- 1 **for** $t = 1, \dots, T$ **do**
 - 2 The learner obtains predictions from N experts;
 - 3 The learner chooses a distribution $p_t \in \Delta(N)$;
 - 4 The learner samples one expert $i_t \sim p_t$;
 - 5 The environment gives the learner the true value and the loss/error of all experts: $\ell_t \in \{0, 1\}^N$;
 - 6 The learner suffers expected loss $\sum_{i \in [N]} p_t(i) \ell_t(i) = \langle p_t, \ell_t \rangle$;
-

Linear function

With the continuous action space, it is easy to reduce the expert problem to an OCO: see Algorithm 7
(The problem is convex since the loss function is convex and the parameter space $\Delta(N)$ is convex.)

Algorithm 7: The expert problem

- 1 **for** $t = 1, \dots, T$ **do**
 - 2 The learner obtains predictions from N experts;
 - 3 The learner invokes the OCO oracle to obtain $p_t \in \Delta(N)$;
 - 4 The learner chooses to follow prediction of one of the experts $i_t \in [N]$;
 - 5 The environment gives the learner the true value. The learner is thus able to learn the loss of each of the experts: $\ell_t \in \{0, 1\}^N$;
 - 6 The learner suffers the loss of the expert which was chosen: $\ell_t(i_t)$. The learner can reconstruct the loss function $f_t(p) = \langle p, \ell_t \rangle$ and give it to the OCO oracle.
-

Cont-action space

← $i_t \sim p_t$

definition

In this setting, one can rewrite the regret as:

$$\text{authors } \ell_t(i) \xrightarrow{\text{reconstruct loss function}} \text{Regret} = \sum_{t=1}^T \langle p_t, \ell_t \rangle - \underbrace{\min_{i \in [N]} \sum_{t=1}^T \ell_t(i)}_{\text{best loss}} \quad (11.7)$$

$$= \sum_{t=1}^T \langle p_t, \ell_t \rangle - \min_{p \in \Delta(N)} \sum_{t=1}^T \langle p, \ell_t \rangle \quad (11.8)$$

$$= \sum_{t=1}^T f_t(p_t) - \min_{p \in \Delta(N)} \sum_{t=1}^T f_t(p). \quad (11.9)$$

We obtain (11.8) because

$$\sum_{t=1}^T \langle p, \ell_t \rangle = \left\langle p, \sum_{t=1}^T \ell_t \right\rangle \geq \min_{i \in [N]} \left[\sum_{t=1}^T \ell_t(i) \right], \quad (11.10)$$

with equality for the probability distribution $p(i) = 1$ when $i = \operatorname{argmin}_i \left[\sum_{t=1}^T \ell_t(i) \right]$ and $p(i) = 0$ otherwise, and (11.9) is by definition of f_t .

11.4 Reducing online learning to (batch learning) traditional learning method.

In this section, we present a reduction from online learning to standard supervised learning problem, also known as the “batch problem” in this literature.

As in the standard supervised learning setting, consider an i.i.d dataset $\{(x_t, y_t)\}_{t=1}^T$ and some parameter w . Let $L(w)$ and $\hat{L}(w)$ be the population loss and empirical loss respectively. For simplicity, assume $|\ell((x_i, y_i), w)| \leq 1$. The theorem below establishes a link between the regret obtained in online learning and the excess risk obtained in the batch setting.

Theorem 11.3 (Relationship between batch excess risk and online regret). Assume $\ell((x, y), w)$ is convex. Suppose we run an online learning algorithm on the dataset $\{(x_i, y_i)\}_{i=1}^T$ and obtain a sequence of models $w_1, \dots, w_T$, and regret R_T . Let $\bar{w} = \frac{1}{T} \sum_{i=1}^T w_i$, then the excess risk of $\bar{w}$ can be bounded above: from batch

$$\underbrace{\frac{L(\bar{w})}{\text{excess risk}}} - \underbrace{\frac{L(w^*)}{\text{optimal}}} \leq \underbrace{\frac{R_T}{T}}_{\text{regret}} + \tilde{O}\left(\frac{1}{\sqrt{T}}\right), \quad (11.11)$$

where $w^* = \operatorname{argmin}_{w \in \Omega} L(w)$.

Here are some intuitive interpretations of the theorem:

- If $R_T = O(T)$, then we have some non-trivial result. Otherwise, the bound in (11.11) is increasing T and does not provide any useful information. $R_T/T = \text{const}$, $\Rightarrow$ Bound
- If the batch problem has a $1/\sqrt{T}$ generalization bound, then the best you can hope for in online learning is $R_T = O(\sqrt{T})$. $\Rightarrow \frac{R_T}{T} \rightarrow 0$ converge to zero.
- If the batch problem has a $1/T$ generalization bound, you can hope for $O(1)$ regret (or $\tilde{O}(1)$ regret in some cases).
- We often have $O(1/\sqrt{T})$ excess risk supervised learning problems; hence it is reasonable to expect $O(\sqrt{T})$ regret in online learning problems.

$$\frac{R_T}{T} \approx O\left(\frac{1}{\sqrt{T}}\right) \Rightarrow R_T \approx O(\sqrt{T})$$

11.5 Follow-the-Leader (FTL) algorithm

In this section, we analyze an algorithm called “Follow-the-Leader” (FTL) for OCO, which is intuitive but fails to perform well in many cases.

The FTL algorithm behaves as its name suggests: it always selects the action w_t such that it minimizes the historical loss the learner has seen so far, i.e.

$$w_t = \operatorname{argmin}_{w \in \Omega} \sum_{i=1}^{t-1} f_i(w). \quad \begin{array}{l} \text{1~t-1 중 best!} \\ \rightarrow w_t \text{에 사용.} \end{array} \quad (11.12)$$

We now demonstrate how the FTL algorithm can fail for the expert problem. In the expert problem, $f_t(p) = \langle p, \ell_t \rangle$, so

$$p_t = \operatorname{argmin}_{p \in \Delta(N)} \sum_{i=1}^{t-1} f_i(p) \quad (11.13)$$

$$= \operatorname{argmin}_{p \in \Delta(N)} \sum_{i=1}^{t-1} \langle \ell_i, p \rangle \quad (11.14)$$

$$= \operatorname{argmin}_{p \in \Delta(N)} \left\langle \sum_{i=1}^{t-1} \ell_i, p \right\rangle. \quad (11.15)$$

The minimizer $p \in \Delta(N)$ is a point-mass probability, with the point mass at the smallest coordinate of $\sum_{i=1}^{t-1} \ell_i$. This gives regret

$$\text{Regret} = \sum_{i=1}^{t-1} \ell_i(i_t), \quad \text{where } i_t = \operatorname{argmin}_{j \in [N]} \sum_{i=1}^{t-1} \ell_i(j). \quad \text{FTL} \quad (11.16)$$

Now, consider the following example: suppose we have only two experts. Suppose expert 1 makes perfect predictions on even days while expert 2 makes perfect predictions on odd days. Assume also that the FTL algorithm chooses expert 1 to break ties (this is not an important point but makes the exposition simpler.) In this setting, the FTL algorithm always selects the *wrong* expert to follow. A few rounds of simulation of this example is shown in Table 11.1

Table 11.1: An example where FTL fails

expert	1	2
true	even	odd

Day	1	2	3	4	...	...
Expert 1's loss	1	0	1	0	...	...
Expert 2's loss	0	1	0	1	...	...
FTL choice i_t	1	2	1	2	1	...

(t-1)까지 더한걸 고르다
매번 틀리게 됨.

실제 양자.

The best expert in hindsight has a loss of $T/2$ (choosing either expert all the time incurs this loss, and so the regret of the FTL algorithm is $T - T/2 = T/2 = \Theta(T)$). The main reason for FTL's failure is that is a deterministic algorithm driven by an extreme update, with no consideration on potential domain shift (it always selects the best expert based on the past with no consideration of the potential next f_t). Knowing its deterministic strategy, the environment can easily play in an adversarial manner. To perform better in a problem like this, we need some “randomness” to hedge risk. ★

11.6 Be-the-leader (BTL) algorithm

A better strategy is called “Be the Leader” (BTL). At time t , the BTL strategy chooses the action that would have performed best on $f_1, \dots, f_{t-1}$ and f_t . In other words, the BTL action at time t is w_{t+1} , as

선택한 행

defined for the FTL algorithm. Note that this is an “illegal” choice for the action because w_{t+1} depends on f_t : in online convex optimization, the action at time t is required to be chosen *before* seeing the function f_t . Nevertheless, we can still gain some useful insights by analyzing this procedure. In particular, the following lemma shows that the BTL strategy is worth emulating because it achieves very good regret.

Lemma 11.4. *The BTL strategy has non-positive regret. That, is, if w_t is defined as in the FTL algorithm, then*

$$BTL \text{ regret} = \sum_{t=1}^T f_t(w_{t+1}) - \min_{w \in \Omega} \sum_{t=1}^T f_t(w) \leq 0, \quad (11.17)$$

for any T and any sequence of functions $f_1, \dots, f_T$.

Proof. We prove the lemma by induction on T . (11.17) holds trivially for $T = 1$. Suppose that (11.17) holds for all $t \leq T - 1$ and any $f_1, \dots, f_{T-1}$. Now we wish to extend (11.17) to time $t = T$. Let f_T be any function. Since $w_{T+1} = \operatorname{argmin}_w \sum_{t=1}^T f_t(w)$, we can write:

$$\sum_{t=1}^T f_t(w_{t+1}) - \min_{w \in \Omega} \sum_{t=1}^T f_t(w) = \sum_{t=1}^T f_t(w_{t+1}) - \sum_{t=1}^T f_t(w_{T+1}) \quad (11.18)$$

$$= \sum_{t=1}^{T-1} f_t(w_{t+1}) - \sum_{t=1}^{T-1} f_t(w_{T+1}) \quad \begin{array}{l} \text{vanished } T\text{-term.} \\ \text{(final summands cancel)} \end{array} \quad (11.19)$$

$$\leq \sum_{t=1}^{T-1} f_t(w_{t+1}) - \min_{w \in \Omega} \sum_{t=1}^{T-1} f_t(w) \quad (11.20)$$

$$\leq 0. \quad \text{(induction hypothesis)} \quad (11.21)$$

□

A useful consequence of this lemma is a regret bound for the FTL strategy.

Lemma 11.5. (FTL regret bound) *Again, let w_t be as in the FTL algorithm. The FTL strategy has the regret guarantee*

$$FTL \text{ regret} = \sum_{t=1}^T f_t(w_t) - \min_{w \in \Omega} \sum_{t=1}^T f_t(w) \leq \sum_{t=1}^T [f_t(w_t) - f_t(w_{t+1})]. \quad (11.22)$$

Proof.

$$FTL \text{ regret} = \sum_{t=1}^T f_t(w_t) - \min_{w \in \Omega} \sum_{t=1}^T f_t(w) \quad (11.23)$$

$$= \sum_{t=1}^T f_t(w_{t+1}) - \min_{w \in \Omega} \sum_{t=1}^T f_t(w) + \sum_{t=1}^T [f_t(w_t) - f_t(w_{t+1})] \quad (11.24)$$

$$\leq 0 + \sum_{t=1}^T [f_t(w_t) - f_t(w_{t+1})], \quad (11.25)$$

where the last inequality is due to (11.17).

□

Lemma 11.5 tells us that if terms $f_t(w_t) - f_t(w_{t+1})$ are small (e.g. w_t does not change much from round to round), then the FTL strategy can have small regret. It suggests that the player should adopt a stable policy, i.e. one where the terms $f_t(w_t) - f_t(w_{t+1})$ are small. It turns out that following this intuition will lead to a strategy that improves the regret all the way to $O(\sqrt{T})$ in certain cases.

FTL : 과거에 많은 의존.

⇒ 과거를 조금만 의존. extremely update 리스크.
: Regularization.

11.7 Follow-the-regularized-leader (FTRL) strategy

Now, we discuss a OCO strategy aims to improve the stability of FTL by controlling the differences $f_t(w_t) - f_t(w_{t+1})$. To describe the method, we will first need a preliminary definition.

Definition 11.6. We say that a differentiable function $\phi : \Omega \mapsto \mathbb{R}$ is α -strongly-convex with respect to the norm $\|\cdot\|$ on Ω if we have

$$\phi(x) \geq \phi(y) + \langle \nabla \phi(y), x - y \rangle + \frac{\alpha}{2} \|x - y\|^2 \quad (11.26)$$

for any $x, y \in \Omega$.

Remark 11.7. If ϕ is convex, then we know that $\phi(x)$ has a linear lower bound $\phi(y) + \langle \nabla \phi(y), x - y \rangle$. Being α -strong-convex means that $\phi(x)$ has a quadratic lower bound, the RHS of (11.26). This quadratic lower bound is very useful in proving theorems in optimization.

Remark 11.8. If $\nabla^2 \phi(y) \succeq \alpha I$ for all y , then ϕ is α -strongly-convex. This follows directly from writing the second-order Taylor expansion of ϕ around y .

Given a 1-strongly-convex function $\phi(\cdot)$, which we call a *regularizer*, we can implement the “Follow the Regularized Leader” (FTRL) strategy. At time t , this strategy chooses the action

$$w_t = \operatorname{argmin}_{w \in \Omega} \left[\sum_{i=1}^{t-1} f_i(w) + \frac{1}{\eta} \phi(w) \right], \quad (11.27)$$

where $\eta > 0$ is a tuning parameter that we will tune later.

11.7.1 Regularization and stability

To understand why we might use the FTRL policy, we first establish that it achieves the intended goal of controlling the differences $f_t(w_t) - f_t(w_{t+1})$. Actually, we will show a more general result that adding a regularizer induces stability for any convex objective.

Lemma 11.9. (Regularizers induce stability) Let F and f be functions taking Ω into $\mathbb{R}$, and assume that F is α -strongly-convex with respect to the norm $\|\cdot\|$ and that f is convex. Let $w = \operatorname{argmin}_{z \in \Omega} F(z)$ and $w' = \operatorname{argmin}_{z \in \Omega} [f(z) + F(z)]$. Then

$$0 \leq f(w) - f(w') \leq \frac{1}{\alpha} \|\nabla F(w)\|_*^2, \quad (11.28)$$

where $\|\cdot\|_*$ is the dual norm of $\|\cdot\|$.

Proof. By strong convexity,

$$\text{for } F, \text{ minima } w. \quad F(w') - F(w) \geq \langle \nabla F(w), w' - w \rangle + \frac{\alpha}{2} \|w - w'\|^2 \quad (11.29)$$

$$\geq \frac{\alpha}{2} \|w - w'\|^2, \quad \text{KKT : } \langle \nabla F(w), w' - w \rangle \geq 0 \quad (11.30)$$

where in the second step we used the fact that the KKT optimality conditions for w imply $\langle \nabla F(w), w' - w \rangle \geq 0$. (Informally, if $\Omega = \mathbb{R}^d$, then $\nabla F(w) = 0$ as w minimizes F . If Ω is a convex subset of $\mathbb{R}^d$, then the gradient $\nabla F(w)$ must be perpendicular to the tangent to Ω at w ; otherwise, we could move in the direction of the negative gradient and project back to the set Ω to lower the value of F .) Since $F + f$ is also α -strongly convex, exactly the same argument implies:

$$\text{for } F+f, \text{ minima } w' \quad [F(w) + f(w)] - [F(w') + f(w')] \geq \frac{\alpha}{2} \|w - w'\|^2. \quad (11.31)$$

(11.30) + (11.31)

Adding these two inequalities gives

$$f(w) - f(w') \geq \alpha \|w - w'\|^2. \quad (11.32)$$

Since this lower bound is clearly positive, this shows $0 \leq f(w) - f(w')$.

Next, we prove the upper bound on $f(w) - f(w')$. Rearranging the inequality (11.32), we obtain

$$\|w - w'\| \leq \sqrt{\frac{1}{\alpha} [f(w) - f(w')]} \quad (11.33)$$

Since f is convex, we have $f(w') \geq f(w) + \langle \nabla f(w), w' - w \rangle$. Rearranging this gives

$$\begin{aligned} f(w) - f(w') &\leq \langle \nabla f(w), w - w' \rangle \\ &\leq \|\nabla f(w)\|_* \cdot \|w - w'\| && \text{(by Cauchy-Schwarz)} \\ &\leq \|\nabla f(w)\|_* \sqrt{\frac{1}{\alpha} [f(w) - f(w')]} && \text{(by (11.33))} \end{aligned}$$

Since $f(w) - f(w') \geq 0$, we can square both sides of this inequality to conclude that

$$[f(w) - f(w')]^2 \leq \|\nabla f(w)\|_*^2 \frac{1}{\alpha} [f(w) - f(w')]. \quad (11.34)$$

Dividing both sides of this expression by $f(w) - f(w')$ gives the desired upper bound. $\square$

Remark 11.10. Consider the special case where $\nabla f(w) = 0$. In this situation w is the minimizer of both F and f , and hence is the minimizer of $F + f$. This implies that $w = w'$, and the inequalities in (11.28) become equalities.

11.7.2 Regret of FTRL

We are now ready to prove a regret bound for the FTRL procedure, based on the idea that strongly convex regularizers induce stability.

Theorem 11.11. (Regret of FTRL) Let ϕ be a 1-strongly-convex regularizer with respect to the norm $\|\cdot\|$ on Ω . Then the FTRL algorithm (11.27) satisfies the regret guarantee

$$\text{FTRL regret} = \sum_{t=1}^T f_t(w_t) - \argmin_{w \in \Omega} \sum_{t=1}^T f_t(w) \leq \frac{D}{\eta} + \eta \sum_{t=1}^T \|\nabla f_t(w_t)\|_*^2, \quad (11.35)$$

where $D = \max_{w \in \Omega} \phi(w) - \min_{w \in \Omega} \phi(w)$. $\star$

Remark 11.12. Suppose that for all t and w , we have the uniform bound $\|\nabla f_t(w)\|_* \leq G$. Then Theorem 11.11 implies that the regret is upper bounded by $D/\eta + \eta GT$. Optimizing this upper bound over η by

taking $\eta = \sqrt{\frac{D}{TG^2}}$ gives the guarantee

$\star$ 1-clip by η ; $-\frac{D}{\eta^2} + \eta GT = 0$

$$\text{FTRL regret} \leq 2\sqrt{DG} \times \sqrt{T} = O(\sqrt{T}) \quad (11.36)$$

In other words, optimally-tuned FTRL can achieve $O(\sqrt{T})$ regret in many cases.

Proof. For convenience, define $f_0(w) = \phi(w)/\eta$. Then the FTRL policy can be written as

$$w_t = \argmin_{w \in \Omega} \sum_{i=0}^{t-1} f_i(w), \quad (11.37)$$

i.e. FTRL is just FTL with an additional “round” of play at time zero. Thus, by Lemma 11.5 with time starting from $t = 0$, we have

09/11/21.

$$\sum_{t=0}^T f_t(w_t) - \operatorname{argmin}_{w \in \Omega} \sum_{t=0}^T f_t(w) \leq \sum_{t=0}^T [f_t(w_t) - f_t(w_{t+1})]. \quad (11.38)$$

For any $t \geq 1$, applying Lemma 11.9 with $F(w) = \sum_{i=0}^{t-1} f_i(w)$ (which is $1/\eta$ -strongly-convex) and $f(w) = f_t(w)$ gives the bound $f_t(w_t) - f_t(w_{t+1}) \leq \eta \|\nabla f_t(w_t)\|_*^2$. Plugging this into the preceding display gives the upper bound:

$$\sum_{t=0}^T f_t(w_t) - \operatorname{argmin}_{w \in \Omega} \sum_{t=0}^T f_t(w) \leq \underbrace{f_0(w_0) - f_0(w_1)}_{=0} + \eta \sum_{t=1}^T \|\nabla f_t(w_t)\|_*^2. \quad (11.39)$$

Next, we need to relate the LHS of the above display (which starts at time $t = 0$) to the actual regret of FTRL (which starts at time $t = 1$). To do this, define $w^* = \operatorname{argmin}_{w \in \Omega} \sum_{t=1}^T f_t(w)$. Then,

$$\sum_{t=0}^T f_t(w_t) - \operatorname{argmin}_{w \in \Omega} \sum_{t=0}^T f_t(w) \geq \sum_{t=0}^T f_t(w_t) - \sum_{t=0}^T f_t(w^*) \quad (11.40)$$

$$= f_0(w_0) - f_0(w^*) + \underbrace{\left(\sum_{t=1}^T f_t(w_t) - \operatorname{argmin}_{w \in \Omega} \sum_{t=1}^T f_t(w) \right)}_{\text{Regret of FTRL}}. \quad (11.41)$$

Combining this inequality with (11.39) gives

$$\text{Regret of FTRL} \leq \cancel{f_0(w_0)} - f_0(w_1) + f_0(w^*) - \cancel{f_0(w_0)} + \eta \sum_{t=1}^T \|\nabla f_t(w_t)\|_*^2 \quad (11.42)$$

$$= \frac{\phi(w^*) - \phi(w_1)}{\eta} + \eta \sum_{t=1}^T \|\nabla f_t(w_t)\|_*^2 \quad \text{here } \phi = \eta \phi \quad (11.43)$$

$$\leq \frac{D}{\eta} + \eta \sum_{t=1}^T \|\nabla f_t(w_t)\|_*^2. \quad \Rightarrow \text{regularizer is 09/11/21.} \quad (11.44)$$

This concludes the proof of the theorem. $\square$

11.7.3 Applying FTRL to online linear regression

We apply the FTRL algorithm to a concrete machine learning problem. Let $\Omega = \{\omega : \|\omega\|_2 \leq 1\}$, and let $\Rightarrow f_t(\omega) = \frac{1}{2}(y_t - \omega^\top x_t)^2$ for some observation pair (x_t, y_t) satisfying $\|x_t\|_2 \leq 1$ and $|y_t| \leq 1$. This corresponds to a problem where we are trying to make accurate predictions using a linear model, but we do not assume any structure on the observation sequence (x_t, y_t) beyond boundedness.

Consider using FTRL in this problem with a ridge regularizer, $\phi(\omega) = \frac{1}{2}\|\omega\|_2^2$. One can check that ϕ is 1-strongly-convex with respect to the ℓ_2 -norm, and also that $D = \max_{\omega \in \Omega} \phi(\omega) - \min_{\omega \in \Omega} \phi(\omega) = \frac{1}{2}$. Moreover, for all t and w we have

$$\nabla f_t(w) = -(y_t - w^\top x_t)x_t, \quad (11.45)$$

$$\|\nabla f_t(w)\|_2 \leq |y_t - w^\top x_t| \cdot \|x_t\|_2 \quad (11.46)$$

$$\leq 2 \cdot 1 = 2. \quad (11.47)$$

parameter 生成法

Therefore, by choosing $\eta = \sqrt{1/(8T)}$ and applying the FTRL regret theorem (Theorem 11.11), we can obtain the regret guarantee

$$\sum_{t=1}^T (y_t - w_t^\top x_t)^2 - \min_{\|w\|_2 \leq 1} \sum_{t=1}^T (y_t - w^\top x_t)^2 \leq 4\sqrt{T} = O(\sqrt{T}) \quad (11.48)$$

11.7.4 Applying FTRL to the expert problem

For the expert problem, recall that the action space is $\Delta(N)$ and $f_t = \langle \ell_t, p \rangle$, where $\ell_t \in [0, 1]^N$. As a first attempt at applying FTRL to this problem, we set $\phi(p) = \frac{1}{2} \|p\|_2^2$. With this choice,

① $\hookrightarrow$ regularizer.

$$D = \max_{p \in \Delta(N)} \phi(p) - \min_{p \in \Delta(N)} \phi(p) \quad (11.49)$$

$$\leq \max_{p \in \Delta(N)} \frac{1}{2} \|p\|_2^2 \quad (11.50)$$

$$\leq \max_{p \in \Delta(N)} \frac{1}{2} \|p\|_1^2 \quad \text{here } p \in [0, 1] \quad (11.51)$$

$$= \frac{1}{2}. \quad (11.52)$$

Also,

$$\|\nabla f_t\|_2 = \|\ell_t\|_2 \leq \sqrt{N}. \quad (11.53)$$

Thus, the regret bound is $O(G\sqrt{DT}) = O(\sqrt{NT})$. This is optimal dependency on T , but not good dependency on N .

Next, we show that if we change our regularization, we can get a better regret guarantee which is logarithmic in N , i.e., the regret is $O(\sqrt{(\log N) \cdot T})$. The new regularizer we choose is the (negative) entropy regularizer:

$$\textcircled{2} \text{ Entropy-regularizer. } \phi(p) = -H(p) = -\sum_{j=1}^N p(j) \log p(j), \quad (11.54)$$

where $p \in \Delta(N)$ is in the set of distributions over $[N]$. We first introduce the following nice property of this regularizer:

Lemma 11.13. $\phi(p)$ defined above is 1-strongly convex with respect to the ℓ_1 norm $\|\cdot\|_1$.

Proof. By definition of strong convexity, we need to show that for all $p, q \in \Delta(N)$,

$$\phi(p) - \phi(q) - \langle \nabla \phi(q), p - q \rangle \geq \frac{1}{2} \|p - q\|_1^2. \quad (11.55)$$

From direct computation, we know the gradient of $\phi(q)$ is

$$\nabla \phi(q) = \begin{bmatrix} 1 + \log q(1) \\ \vdots \\ 1 + \log q(N) \end{bmatrix}. \quad (11.56)$$

Plugging this into the LHS of (11.55), we get

$$\phi(p) - \phi(q) - \langle \nabla \phi(q), p - q \rangle \quad \leftarrow \text{11.56 대칭.} \quad (11.57)$$

$$= \sum_{j=1}^N p(j) \log p(j) - \sum_{j=1}^N q(j) \log q(j) - \sum_{j=1}^N (1 + \log q(j)) (p(j) - q(j)) \quad (11.58)$$

$$= \sum_{j=1}^N p(j) \log p(j) - \sum_{j=1}^N p(j) \log q(j) - \sum_{j=1}^N (p(j) - q(j)) \quad (11.59)$$

$$= \sum_{j=1}^N p(j) \log \frac{p(j)}{q(j)} \quad (11.60)$$

$$= KL(p||q), \quad (11.61)$$

where $KL(p||q)$ is the **KL-divergence between p and q** . (We used the fact that $\sum_{j=1}^N p(j) = \sum_{j=1}^N q(j) = 1$ to get (11.60).) Finally, we finish the proof by applying **Pinsker's inequality**: $KL(p||q) \geq \frac{1}{2} \|p - q\|_1^2$. $\square$

Hence, ϕ satisfies the condition on the regularizer for our FTRL regret guarantee. To obtain the regret bound (11.36), we also need to bound $D = \sup \phi(p) - \inf \phi(p)$ and $G = \sup \|\nabla f_t(w)\|_\infty$ (since $\|\cdot\|_\infty$ is the dual norm of $\|\cdot\|_1$). Since negative entropy is always non-positive and (positive) entropy is always bounded above by $\log N$, we bound D with

$$\text{11.54 : Shannon entropy} \quad D = \sup \phi(p) - \inf \phi(p) \leq -\inf \phi(p) = -\inf(-H(p)) = \sup(H(p)) \leq \log N, \quad (11.62)$$

and we bound G with

$$\text{Regret}_{\text{FTRL}} \leq 2\sqrt{GT} \quad G = \|\nabla f_t(w)\|_\infty = \|l_t\|_\infty \leq 1. \quad (11.63)$$

Plugging these two into the regret bound (11.36) we get bound $O(\sqrt{(\log N) \cdot T})$.

Thus far, we have looked at FTRL and the expert problem abstractly: at each time t we choose action p_t based on the update

$$p_t = \operatorname{argmin}_{p \in \Delta(N)} \sum_{i=1}^{t+1} f_i(p) - \frac{1}{\eta} H(p). \quad (11.64)$$

Can we get an exact analytical solution for p_t ? Since we are minimizing a convex function, we can call some off-the-shelf convex optimization algorithm to solve this at each step. Another way is to write down the KKT conditions and solve that set of equations. Instead, we will show that there exists much simpler ways to solve this update. In particular, we will be using the *Gibbs variational principle*, which is essentially the KKT conditions under the hood.

Lemma 11.14 (Gibbs variational principle). *Let ν, μ be probability distributions on $[N]$. Then*

$$\sup_{\nu} \left(\mathbb{E}_{\nu}[f] - KL(\nu||\mu) \right) = \log \mathbb{E}_{\mu}[e^f], \quad \Rightarrow \text{가장 높은 기대값을 갖는 분포를 찾는 것, KL 문제.} \quad (11.65)$$

where $\mathbb{E}_{\nu}[f] = \mathbb{E}_{x \sim \nu}[f(x)] = \langle \nu, f \rangle$ and $\mathbb{E}_{\mu}[e^f] = \mathbb{E}_{x \sim \mu}[e^{f(x)}]$. Moreover, the optimal solution is attained at

$$\text{optimal solution} \quad \Rightarrow \nu(x) \propto \mu(x) \cdot e^{f(x)}. \quad (11.66)$$

Intuitively, Lemma 11.14 says that taking the supremum over distributions μ of a linear function plus the KL divergence as the regularizer will give us the same distribution as exponentiating f .

If we take μ to be the uniform distribution on $[N]$ and replace f with $-f$ in Lemma 11.14, we get the following corollary:

minimize $\mathbb{E}[f] - H(\nu)$ subject to $\nu \in \Delta(N)$

Corollary 11.15. Let ν be a probability distribution. Then, $\mathbb{E}_\nu[f] - H(\nu)$ is minimized at $\nu(x) \propto e^{-f(x)}$.

Proof. When μ is uniform distribution, we have

$$KL(\nu||\mu) = \sum_x \nu(x) \log \frac{\nu(x)}{\mu(x)} \quad (11.67)$$

$$= \log N - \sum_x \nu(x) \log \frac{1}{\nu(x)} \quad (11.68)$$

$$= \log N - H(\nu). \quad (11.69)$$

So $\sup_\nu (\mathbb{E}_\nu[-f] - KL(\nu||\mu)) = -\inf_\nu (\mathbb{E}_\nu[f] - H(\nu) + \log N)$. This means that the value of ν that attains the infimum of $\mathbb{E}_\nu[f] - H(\nu)$ is the same ν attaining the supremum of $\mathbb{E}_\nu[-f] - KL(\nu||\mu)$, which by Lemma 11.14 is proportional to $e^{-f(x)}$. $\square$

We now apply the Gibbs variational principle to the expert problem. Notice that our FTRL update for the expert problem at time t can be written as

$$\operatorname{argmin}_{p_t \in \Delta(N)} \left\langle \sum_{i=1}^{t-1} l_i, p_t \right\rangle - \frac{1}{\eta} H(p_t) = \operatorname{argmin}_{p_t \in \Delta(N)} \left\langle \eta \sum_{i=1}^{t-1} l_i, p_t \right\rangle - H(p_t), \quad (11.70)$$

where l_i is the vector of expert losses at time i . Letting $f = \eta \sum_{i=1}^{t-1} l_i$, we know from Corollary 11.15 that the minimizer is attained at $p_t \propto \exp\left(-\eta \sum_{i=1}^{t-1} l_i\right)$, or equivalently,

$$p_t(j) = \frac{\exp(-\eta L_t(j))}{\sum_{k=1}^N \exp(-\eta L_t(k))}, \quad (11.71)$$

where $L_t = \sum_{i=1}^{t-1} l_i$ is the cumulative loss vector. Basically, solving the expert problem is to look at the historical loss of each expert and take softmax to find the probability distribution of how much to trust each expert.

This algorithm is also called the “Multiplicative Weights Update”, which has been studied before online learning framework became popular [Arora et al. 2005, Freund and Schapire 1997, Littlestone and Warmuth, 1994]. One way of doing multiplicative weights update is the following: Let $\tilde{p}_t$ be the unnormalized distribution that we keep track of. At each time step t , for each expert j , we look at $l_{t-1}(j)$. if $l_{t-1}(j) = 1$, i.e. the expert made a mistake at the previous time step, we update $\tilde{p}_t(j) = \tilde{p}_{t-1}(j) \cdot \exp(-\eta)$; otherwise we make no change. We then get a distribution by normalizing $\tilde{p}_t$:

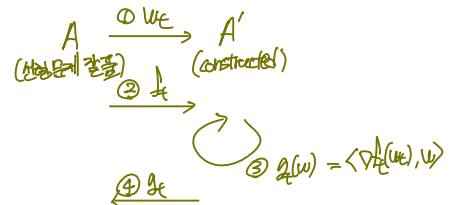
$$p_t = \frac{\tilde{p}_t}{\|\tilde{p}_t\|_1}. \quad (11.72)$$

11.8 Convex to linear reduction 공변량 방법

In the previous section we considered the expert problem, where the loss function is a *linear* function of the parameters. At first glance we may think this is a very restrictive constraint for online convex optimization. However, as we will see in this section, we can always assume f_t to be linear in online convex optimization without loss of generality. That means that for online learning, the linear case is the hardest one.

More concretely, assume we have an algorithm $\mathcal{A}$ that solves the linear case. Given any online convex optimization, we will build an algorithm $\mathcal{A}'$ which invokes algorithm $\mathcal{A}$ in the following fashion: for $t = 1, \dots, T$,

1. The learner invoke $\mathcal{A}$ to get output action $w_t \in \Omega$.
2. The environment gives the learner the loss function $f_t(\cdot)$.



3. The learner constructs a linear function $g_t(w) = \langle \nabla f_t(w_t), w \rangle$, which is the *local linear approximation* of f at w . (Technically the local linear approximation of f and w is $\langle \nabla f_t(w_t), w - w_t \rangle$, but we drop the w_t shift for convenience.)
4. The learner feeds $g_t(\cdot)$ to algorithm $\mathcal{A}$ as the loss function.

We have the following informal claim¹:

Proposition 11.8.1 (Informal). If a deterministic algorithm $\mathcal{A}$ has regret no more than $\gamma(T)$ for linear cases for some function $\gamma(\cdot)$, then $\mathcal{A}'$ stated above has regret no more than $\gamma(T)$ for convex functions.

Proof. For all $w \in \Omega$, the regret guarantee on $\mathcal{A}$ tells us that

$$\sum_{t=1}^T g_t(w_t) - \sum_{t=1}^T g_t(w) \leq \gamma(T). \quad (11.73)$$

Since f_t is convex, we also know that

$$g_t(w_t) - g_t(w) = \langle \nabla f_t(w_t), w_t - w \rangle \geq f_t(w_t) - f_t(w). \quad (11.74)$$

Therefore, for all $w \in \Omega$,

$$\sum_{t=1}^T f_t(w_t) - \sum_{t=1}^T f_t(w) \leq \sum_{t=1}^T g_t(w_t) - \sum_{t=1}^T g_t(w) \quad (11.75)$$

$$\leq \gamma(T). \quad (11.76)$$

Hence, the regret for $\mathcal{A}'$ is upper bounded by $\gamma(T)$ as well. $\square$

11.8.1 Online gradient descent

In this section we combine the FTRL framework with ℓ_2 -regularization and the online-to-linear reduction. The resulting algorithm is *online gradient descent*.

Concretely, given any convex online optimization problem, we first do the online-to-linear reduction, then we use FTRL with ℓ_2 regularization ($\phi(w) = \frac{1}{2}\|w\|_2^2$) to solve the resulting linear case. This gives us the following update:

$$w_t = \operatorname{argmin}_{w \in \Omega} \sum_{i=1}^{t-1} g_i(w) + \frac{1}{\eta} \|w\|_2^2 \quad (11.77)$$

$$= \operatorname{argmin}_{w \in \Omega} \sum_{i=1}^{t-1} \langle \nabla f_i(w_i), w \rangle + \frac{1}{\eta} \|w\|_2^2 \quad (11.78)$$

$$= \Pi_{\Omega} \left(-\eta \cdot \sum_{i=1}^{t-1} \nabla f_i(w_i) \right), \quad (11.79)$$

where $\Pi_{\Omega}(\cdot)$ is the projection operator onto the set Ω . The last equality is because for any vector a , we have

$$\operatorname{argmin}_{w \in \Omega} \langle a, w \rangle + \frac{1}{\eta} \|w\|_2^2 = \operatorname{argmin}_{w \in \Omega} \frac{1}{2\eta} \|w + \eta a\|_2^2 - \eta \|a\|_2^2 \quad (11.80)$$

$$= \operatorname{argmin}_{w \in \Omega} \|w + \eta a\|_2^2 \quad (11.81)$$

$$= \operatorname{argmin}_{w \in \Omega} \|w - (-\eta a)\|_2^2 \quad (11.82)$$

$$= \Pi_{\Omega}(-\eta a). \quad (11.83)$$

¹For rigorous proof, we need additional assumptions and restrictions on f_t, g_t .

Intuitively, we can think of this algorithm as gradient descent with “lazy” projection:

$$u_t = \underbrace{w_{t-1}}_{\text{두번까지 많은 상태로 계속 사용}} - \eta \nabla f_{t-1}(w_{t-1}), \quad (11.84)$$

$$w_t = \Pi_{\Omega}(u_t). \quad (11.85)$$

Similarly, we can define gradient descent with “eager” projection (which can get similar regret bounds):

$$u_t = \underbrace{w_{t-1}}_{\text{두번의 결과물에서 출발}} - \eta \nabla f_{t-1}(w_{t-1}), \quad (11.86)$$

$$w_t = \Pi_{\Omega}(u_t). \quad (11.87)$$

This concludes our discussion of online learning in this course.